

BÁO CÁO THỰC HÀNH LAB 01 CROSS-SITE SCRIPTING

MSSV: 21127645 - Họ tên: Lê Minh

1. Mục tiêu và môi trường thực hành

Mục tiêu là phân biệt Reflected, Stored và DOM-based XSS; theo dõi dữ liệu không tin cậy tới sink; kiểm chứng escaping, sanitization, textContent, CSP và cookie flags.

Môi trường: Flask/SQLite và Chrome hoặc Firefox tại 127.0.0.1:5000. Payload chỉ hiển thị alert kiểm thử trong lab local; không kết nối website thật.

2. Kịch bản và các bước thực hiện

Reflected: gửi cùng q vào bản vulnerable và secure. Stored: reset database, POST bình luận, reload rồi mở bản secure. DOM: đặt cùng fragment vào hai trang và quan sát DOM. Cuối cùng đối chiếu code, CSP/cookie và kiểm thử.

ẢNH 01/10

Tên file bắt buộc: 01_reflected_vulnerable.png

Tiêu đề ảnh: Reflected XSS ở bản vulnerable

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5000/vulnerable/search`

Trạng thái: Mở bản vulnerable, xóa kết quả tìm kiếm cũ. Thao tác: nhập `<img src=x onerror="alert('XSS')">`; Bấm Tìm kiếm.

Panel/DevTools: DevTools Network > request search hoặc Request Inspector tích hợp.

Nội dung bắt buộc phải thấy: Thanh địa chỉ localhost, query/input, GET request và alert/verdict payload_executed.

Kết quả mong đợi: Payload được phản chiếu không escape và chạy trong trình duyệt.

Caption: Reflected XSS xảy ra khi dữ liệu q được đưa thẳng vào HTML.

Hình 1. Reflected XSS xảy ra khi dữ liệu q được đưa thẳng vào HTML.

ẢNH 03/10

Tên file bắt buộc: 03_stored_submit.png

Tiêu đề ảnh: Gửi và lưu Stored XSS

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5000/vulnerable/post/1/comments`

Trạng thái: Chạy python scripts/reset_database.py rồi mở trang bình luận vulnerable. Thao tác: nhập Tác giả: SinhVien; nội dung: `<img src=x onerror="alert('XSS')">`; Bấm Gửi bình luận.

Panel/DevTools: Request Inspector và Database Inspector tích hợp (ghép cùng UI nếu đọc được).

Nội dung bắt buộc phải thấy: POST body đã che cookie và bản ghi comments chứa payload trong SQLite.

Kết quả mong đợi: Server lưu dữ liệu thật; việc dùng SQL tham số không tự ngăn XSS.

Caption: Payload Stored XSS được gửi bằng POST và lưu trong cơ sở dữ liệu.

Hình 3. Payload Stored XSS được gửi bằng POST và lưu trong cơ sở dữ liệu.

ẢNH 04/10

Tên file bắt buộc: 04_stored_reload.png

Tiêu đề ảnh: Stored XSS chạy lại sau reload

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5000/vulnerable/post/1/comments`

Trạng thái: Giữ bản ghi của ảnh 03 và đóng alert. Thao tác: nhập Không nhập thêm dữ liệu.; Reload trang (hoặc mở lại bằng phiên local khác).

Panel/DevTools: Final Verdict/Timeline tích hợp.

Nội dung bắt buộc phải thấy: Bình luận đã lưu và alert/verdict payload_executed sau reload.

Kết quả mong đợi: Payload chạy lại mỗi khi người dùng xem trang chứa bản ghi.

Caption: Stored XSS tiếp tục chạy từ dữ liệu đã lưu sau khi tải lại trang.

Hình 4. Stored XSS tiếp tục chạy từ dữ liệu đã lưu sau khi tải lại trang.

ẢNH 06/10

Tên file bắt buộc: 06_dom_vulnerable.png

Tiêu đề ảnh: DOM-based XSS ở innerHTML

Chèn ảnh tại vị trí này.

URL hoặc lệnh:

`http://127.0.0.1:5000/vulnerable/dom-search#%3Cimg%20src%3Dx%20onerror%3Dalert('XSS')%3E`

Trạng thái: Mở trang DOM vulnerable, xóa fragment cũ. Thao tác: nhập Fragment payload an toàn như trong URL.; Nhấn Enter hoặc kích hoạt hashchange.

Panel/DevTools: DOM Inspector/DevTools Elements và bước innerHTML.

Nội dung bắt buộc phải thấy: location.hash, sink innerHTML, element img/onerror và alert/verdict executed.

Kết quả mong đợi: JavaScript phía client tạo DOM nguy hiểm mà fragment không gửi tới server.

Caption: DOM-based XSS phát sinh khi location.hash được gán vào innerHTML.

Hình 6. DOM-based XSS phát sinh khi location.hash được gán vào innerHTML.

3. Nguyên nhân kỹ thuật

Reflected XSS: request.args['q'] được bọc Markup và chèn vào ngữ cảnh HTML nên ký tự đặc biệt không được encode.

Stored XSS: comments.body được lưu trong SQLite rồi đánh dấu an toàn khi render. Lưu bằng SQL tham số chỉ chống SQL Injection, không chống XSS.

DOM-based XSS: JavaScript đọc location.hash và gán vào innerHTML; lỗi xảy ra tại trình duyệt vì fragment không đi trong HTTP request.

Vị trí dữ liệu Reflected là vùng kết quả HTML; bản vulnerable không escape <, >, dấu nháy. Người mở URL payload có thể chạy script trong origin của ứng dụng. Stored nguy hiểm hơn vì tồn tại và có thể tác động mọi người xem; cookie thiếu HttpOnly có thể bị script đọc.

4. Kết quả và bằng chứng

Các ảnh dưới đây đặt cạnh đúng kích bản. Khi PNG hợp lệ tồn tại, generator thay khung hướng dẫn bằng ảnh thật và giữ caption.

ẢNH 02/10

Tên file bắt buộc: 02_reflected_secure.png

Tiêu đề ảnh: Reflected XSS đã được encode

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5000/secure/search`

Trạng thái: Đóng alert, mở bản secure. Thao tác: nhập `<img src=x onerror="alert('XSS')">`; Bấm Tìm kiếm.

Panel/DevTools: Response Inspector hoặc DevTools Elements.

Nội dung bắt buộc phải thấy: Chuỗi `<img...>` hoặc text node, không có element img nguy hiểm, verdict blocked.

Kết quả mong đợi: Jinja autoescape/output encoding làm payload không chạy.

Caption: Output encoding biến payload Reflected XSS thành văn bản an toàn.

Hình 2. Output encoding biến payload Reflected XSS thành văn bản an toàn.

ẢNH 05/10

Tên file bắt buộc: 05_stored_secure.png

Tiêu đề ảnh: Stored XSS đã sanitize/escape

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5000/secure/post/1/comments`

Trạng thái: Giữ dữ liệu DB của ảnh 03, mở bản secure. Thao tác: nhập Không nhập thêm dữ liệu.; Reload trang.

Panel/DevTools: Biến đổi sanitization và DOM/Final Verdict.

Nội dung bắt buộc phải thấy: Dữ liệu gốc, dữ liệu sau Bleach/escape và verdict không thực thi.

Kết quả mong đợi: Event handler nguy hiểm bị loại hoặc encode; nội dung an toàn vẫn hiển thị.

Caption: Sanitization theo allowlist và escaping chặn Stored XSS.

Hình 5. Sanitization theo allowlist và escaping chặn Stored XSS.

ẢNH 07/10

Tên file bắt buộc: 07_dom_secure.png

Tiêu đề ảnh: DOM-based XSS đã dùng `textContent`

Chèn ảnh tại vị trí này.

URL hoặc lệnh:

`http://127.0.0.1:5000/secure/dom-search#%3Cimg%20src%3Dx%20onerror%3Dalert('XSS')%3E`

Trạng thái: Đóng alert và mở trang DOM secure. Thao tác: nhập Củng fragment của ảnh 06.; Nhấn Enter hoặc kích hoạt hashchange.

Panel/DevTools: DOM Inspector/DevTools Elements và bước `textContent`.

Nội dung bắt buộc phải thấy: `Source location.hash`, sink `textContent`, text node, không có `img/onerror` thực thi.

Kết quả mong đợi: Payload hiển thị nguyên văn và không chạy.

Caption: `textContent` loại bỏ khả năng diễn giải fragment thành HTML.

Hình 7. `textContent` loại bỏ khả năng diễn giải fragment thành HTML.

ẢNH 08/10

Tên file bắt buộc: 08_code_fixes.png

Tiêu đề ảnh: Ba nguyên nhân và ba bản vá XSS

Chèn ảnh tại vị trí này.

URL hoặc lệnh: Các tab So sánh mã trong ba trang secure; có thể ghép thành một ảnh.

Trạng thái: Đã tạo trace cho ba kịch bản secure/vulnerable. Thao tác: nhập Không nhập thêm dữ liệu.; Mở So sánh mã ở từng kịch bản.

Panel/DevTools: Code Comparison tích hợp.

Nội dung bắt buộc phải thấy: Markup(q)/autoescape, Markup(row)/Bleach và innerHTML/textContent.

Kết quả mong đợi: Đọc rõ ba sink lỗi và ba thay đổi sửa lỗi.

Caption: So sánh nguyên nhân và bản vá cho ba loại XSS.

Hình 8. So sánh nguyên nhân và bản vá cho ba loại XSS.

ẢNH 09/10

Tên file bắt buộc: 09_csp_cookie.png

Tiêu đề ảnh: CSP và thuộc tính cookie

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5000/security-headers` và `/profile`

Trạng thái: Mở profile local để tạo demo cookie. Thao tác: nhập Không nhập dữ liệu.; Reload hai trang nếu cần.

Panel/DevTools: DevTools Network > Response Headers và Application > Cookies (ghép khi chữ đọc được).

Nội dung bắt buộc phải thấy: Content-Security-Policy; HttpOnly, SameSite, Path và trạng thái Secure local/production.

Kết quả mong đợi: Header/cookie flags hiện rõ; ghi nhận đây là defense in depth, không thay bản vá code.

Caption: CSP và cookie flags giảm tác động nhưng không thay thế sửa sink XSS.

Hình 9. CSP và cookie flags giảm tác động nhưng không thay thế sửa sink XSS.

5. Mức độ ảnh hưởng

Reflected cần nạn nhân mở URL; DOM cần fragment/nguồn client bị kiểm soát; Stored có phạm vi lớn nhất vì payload tồn tại và chạy với nhiều lượt xem. Hậu quả gồm thay đổi DOM, hành động dưới phiên người dùng và rò rỉ dữ liệu mà script truy cập được. HttpOnly giảm rủi ro đọc cookie nhưng không loại bỏ XSS.

6. Bản vá và cách phòng chống

Vulnerable: Markup(q), Markup(row['body']) và `output.innerHTML = value`. Secure: để Jinja autoescape q; sanitize nội dung HTML bằng allowlist rồi vẫn encode đúng ngữ cảnh; dùng `output.textContent = value`.

Biện pháp bổ sung: output encoding theo ngữ cảnh HTML/attribute/URL/JavaScript; sanitization bằng thư viện tin cậy; validate input ở server; không tin URL/form/cookie/localStorage; CSP chặt; cookie HttpOnly, Secure trên HTTPS và SameSite. CSP là defense in depth, không thay việc sửa code.

7. Trả lời các câu hỏi báo cáo trong BaiTapTopic04.docx

So sánh: Reflected lấy payload từ request và phản hồi ngay; Stored lấy từ kho lưu trữ và lặp lại cho người xem; DOM-based do JavaScript client chuyển source như `location.hash` tới sink DOM, có thể không qua server.

Validate input chưa đủ vì dữ liệu hợp lệ ở một ngữ cảnh vẫn nguy hiểm ở ngữ cảnh khác, có nhiều cách biểu diễn/encode và dữ liệu có thể đến từ nguồn ngoài form. Cần output encoding vì nó làm ký tự dữ liệu không còn được parser hiểu là cú pháp thực thi.

CSP không thay sửa code; nó chỉ hạn chế nguồn/thực thi khi lỗi còn tồn tại. Vá từng lỗi: Reflected dùng autoescape/encoding; Stored sanitize allowlist và escape khi render; DOM thay innerHTML bằng textContent hoặc API DOM an toàn.

8. Kết quả kiểm thử

Log pytest thật (dòng cuối): 13 passed in 1.44s

ẢNH 10/10

Tên file bắt buộc: 10_tests_reports.png

Tiêu đề ảnh: Pytest và report artifacts

Chèn ảnh tại vị trí này.

URL hoặc lệnh: Terminal tại thư mục Lab01.

Trạng thái: Đóng ứng dụng khác đang giữ file report. Thao tác: nhập `python -m pytest -q`; `python scripts/generate_report.py`; `Get-ChildItem report`; Chạy lần lượt ba lệnh.

Panel/DevTools: Terminal; không cần DevTools.

Nội dung bắt buộc phải thấy: Dòng tổng kết pytest thực tế, tên DOCX/PDF và kích thước khác 0.

Kết quả mong đợi: Chỉ ghi đạt nếu pytest trả exit code 0; report artifacts tồn tại.

Caption: Kết quả pytest thực tế và các report artifacts của Lab01.

Hình 10. Kết quả pytest thực tế và các report artifacts của Lab01.

9. Kết luận

Ba lỗi cùng bắt đầu từ dữ liệu không tin cậy nhưng khác nơi lưu và nơi thực thi. Bản vá hiệu quả phải loại sink nguy hiểm/encode đúng ngữ cảnh; CSP và cookie flags chỉ tăng chiều sâu phòng thủ.